Building a More Complex Neural Network

We can build a more complex neural network by stacking multiple layers, where the output of one layer becomes the input for the next.

Network Architecture & Layer Counting

- **Example Network:**
  - **Layer 0:** The **Input Layer** ($\vec{x}$)
  - **Layer 1:** Hidden Layer
  - **Layer 2:** Hidden Layer
  - **Layer 3:** Hidden Layer
  - **Layer 4:** The **Output Layer** ($\hat{y}$)
- **Counting Convention:** When we state the number of layers in a network, we count the **hidden layers and the output layer**. We do *not* count the input layer.
  - The example above is a **4-layer neural network**.

Computation Within a Layer (e.g., Layer 3)

Let's look at the computations for **Layer 3**, which takes the activations from Layer 2 ($\vec{a}^{[2]}$) as its input and produces its own activation vector, $\vec{a}^{[3]}$.

If Layer 3 has 3 neurons (or "units"):

- **Neuron 1:** Computes $a_1^{[3]} = g(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]})$
- **Neuron 2:** Computes $a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$
- **Neuron 3:** Computes $a_3^{[3]} = g(\vec{w}_3^{[3]} \cdot \vec{a}^{[2]} + b_3^{[3]})$

The output of this layer is the vector $\vec{a}^{[3]} = [a_1^{[3]}, a_2^{[3]}, a_3^{[3]}]$.

General Formula for a Single Neuron 🧮

We can write a general formula for the computation of a single neuron (unit `j`) in any given layer (`l`):

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

- $a_j^{[l]}$: The activation (output) of the **j-th neuron** in the **l-th layer**.
- $\vec{w}_j^{[l]}$ & $b_j^{[l]}$: The parameters (weight vector and bias) for the **j-th neuron** in the **l-th layer**.

- $\vec{a}^{[l-1]}$: The vector of activations from the **previous layer (l-1)**. This is the input to the current layer.
- g: The **activation function** (so far, we have used the **sigmoid function**).

Consistent Notation

To make this formula work for the very first layer (Layer 1), we introduce one final piece of notation:

- We define the input feature vector, $\vec{x}$, as the "activation" of Layer 0:
  $\vec{a}^{[0]} = \vec{x}$

This allows us to write the computation for Layer 1 as:
$a_j^{[1]} = g(\vec{w}_j^{[1]} \cdot \vec{a}^{[0]} + b_j^{[1]})$

This process of computing the activations layer by layer is the basis for **inference**, or making predictions with a neural network.